

6Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Project Title:

Audio transcription app using open AI whisper

Team Name:

EZ clap

Team Members:

- **Dasari David Samuel**
 - **Adusumalli Deepak**
 - **Addagada Hemanth**
 - **Buragadda Siva Sambaiah**
 - **Siva Sekhara Reddy Annapureddy**
-

Phase-1: Brainstorming & Ideation

Objective:

Problem statement:

Audio transcription app using open AI whisper

Purpose:

An audio transcription app using OpenAI Whisper converts spoken language into written text automatically. It helps improve accessibility for people with hearing impairments, boosts productivity by quickly transcribing meetings, lectures, or interviews, and supports multiple languages. The impact includes saving time and costs, providing high accuracy, making content searchable, and enhancing communication and learning by turning audio into easily accessible text.

Key Points:

1. Problem Statement:

In many industries, converting audio content such as meetings, lectures, interviews, and

podcasts into written text is a time-consuming and labor-intensive process. Manual transcription is often prone to errors and can be expensive. There is a need for an efficient, accurate, and cost-effective solution that can automatically transcribe audio in multiple languages with minimal human intervention. Existing transcription tools may struggle with noisy environments or diverse accents, leading to lower accuracy. This creates a gap for a reliable, AI-powered solution that can quickly and accurately transcribe spoken words into text, making content more accessible, searchable, and useful for various users.

2. Solution:

To address the challenges of manual transcription and existing tools, we can develop an audio transcription app powered by OpenAI Whisper, which offers the following features:

- 1. Automatic Transcription:** The app will use OpenAI Whisper's advanced speech recognition model to convert audio recordings (meetings, lectures, podcasts, etc.) into accurate text with minimal human intervention.
- 2. Multi-Language Support:** Whisper supports transcription in multiple languages, making the app accessible to a global audience and useful in diverse linguistic contexts.
- 3. Noise Resilience:** Whisper's ability to transcribe audio accurately, even in noisy environments or with various accents, ensures high-quality transcriptions in real-world conditions.

3.Target Users:

The Audio Transcription App using OpenAI Whisper will benefit:

- 1. Students and Educators** – For transcribing lectures and study materials.
- 2. Businesses and Professionals** – For transcribing meetings, interviews, and conferences.
- 3. Media and Content Creators** – For transcribing podcasts and videos.
- 4. People with Hearing Impairments** – Making audio content accessible through text.
- 5. Researchers** – For transcribing interviews and focus groups for analysis.
- 6. Global Audiences** – Offering multilingual transcription to reach diverse groups.
- 7. Healthcare Professionals** – For transcribing patient consultations and medical records.

4.Expected outcome:

The expected outcomes of the Audio Transcription App using OpenAI Whisper include:

- 1. Accurate Transcriptions:** Reliable and precise text conversion, even in noisy environments.
 - 2. Increased Accessibility:** Making audio content accessible to people with hearing impairments.
 - 3. Time and Cost Savings:** Faster and cheaper than manual transcription.
 - 4. Multilingual Support:** Transcriptions in multiple languages for a global audience.
 - 5. Improved Productivity:** Quick conversion of spoken content for easier reference.
 - 6. Searchable Text:** Enabling users to search and find specific information in transcriptions.
 - 7. Scalable Solution:** Capable of handling large volumes of audio content efficiently.
-

Phase-2: Requirement Analysis

Objective:

Technical Requirements:

1. **OpenAI Whisper Integration:** Use Whisper API for transcription, supporting various audio formats.
2. **Cloud Infrastructure:** Scalable hosting and storage (AWS, Google Cloud) for audio and transcriptions.
3. **User Interface:** Simple web/mobile interface for uploading files and managing transcriptions.
4. **Real-Time Processing:** Transcription of live audio with minimal delay.

Functional Requirements:

1. **Audio Upload:** Allow file upload in various formats (MP3, WAV, etc.).
2. **Real-Time Transcription:** Transcribe live audio (microphone or streaming).
3. **Language Selection:** Option to select or auto-detect transcription language.

Key Points:

Technical Requirements (Languages, Frameworks, Tools):

1. **Programming Languages:**
 - **Python:** For backend processing with OpenAI Whisper.
 - **JavaScript/TypeScript:** For frontend development (React.js/Vue.js).
2. **Frameworks:**
 - **Flask/Django (Python) or FastAPI:** For backend API.
 - **React.js/Vue.js:** For frontend web development.
3. **Tools:**
 - **OpenAI Whisper:** Core AI model for transcription.
 - **FFmpeg:** For audio file processing.
 - **Socket.io:** For real-time communication.

Functional Requirements:

1. **Audio Upload:** Support multiple file formats.
2. **Real-Time Transcription:** Transcribe live audio.
3. **Language Selection:** Auto-detect or choose language.
4. **Text Editing:** Edit transcriptions after processing.
5. **Export:** Download in formats like TXT, PDF.
6. **History:** Save and access past transcriptions.
7. **Search:** Search through transcriptions.
8. **User Accounts:** Save preferences and transcriptions.

9. Notifications: Alert users on progress/errors.
10. Audio Quality Alerts: Detect poor audio quality.

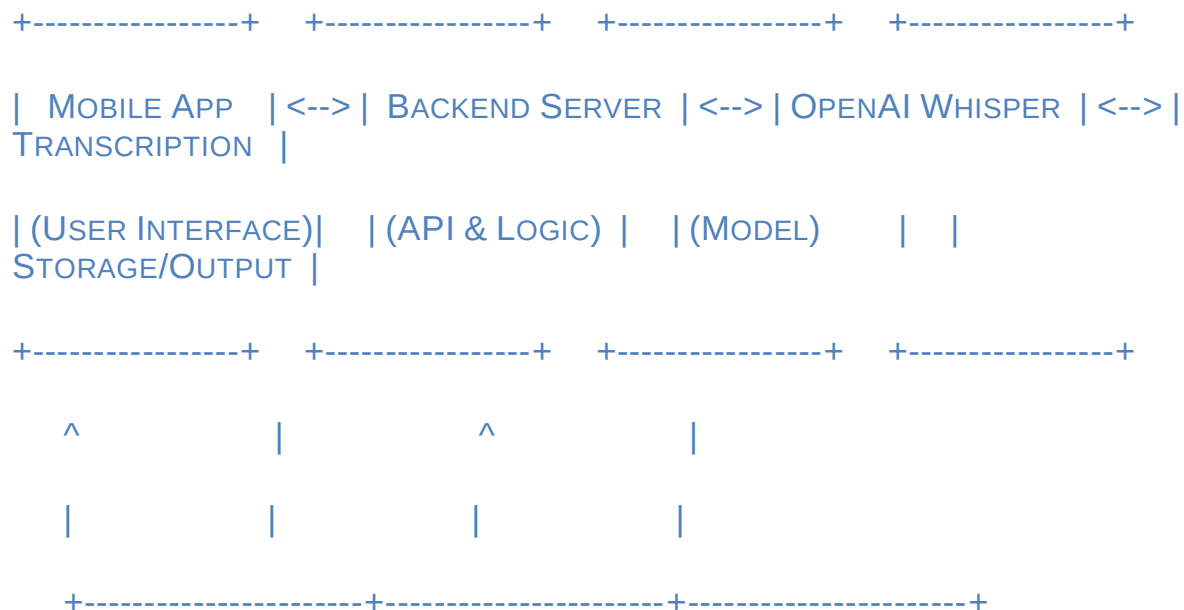
Constraints & Challenges:

-
1. Audio Quality: Poor sound or background noise can affect transcription accuracy.
 2. Language Limitations: Whisper may perform differently across languages and dialects.
 3. Real-Time Processing: Achieving low-latency transcription for live audio is challenging.
 4. Data Privacy: Ensuring secure storage and handling of sensitive data.
 5. Scalability: Managing high volumes of users and audio files without performance issues.
 6. Resource Usage: High computational cost for processing large audio files.
 7. File Size Limits: Large files may exceed upload or processing limits.
 8. Multilingual Support: Transcribing accurately in multiple languages can be difficult.
 9. User Experience: Designing an intuitive interface for complex tasks.
 10. Cost Management: High costs for running and scaling transcription services.

Phase-3: Project Design

Objective:

Architecture:



Phase-4: Project Planning (Agile Methodologies)

Objective:

1. App Setup

- Set up version control (GitHub/GitLab)
- Configure development environment (tech stack, dependencies)
- Organize project structure and initial documentation

2. Audio Upload & Input

- Implement UI for audio file upload
- Create backend endpoint to receive and store files
- Optionally, set up background task queue for processing

3. OpenAI Whisper Integration

- Install Whisper model and integrate it into the backend
- Create API to trigger transcription and handle responses
- Store transcriptions in the database

4. Display Transcriptions

- Build UI to display transcribed text
- Allow users to edit transcriptions and download/export them

5. User Authentication

- Implement login/signup system
- Store transcription history in user profiles
- Optional: Add user settings (language, storage preferences)

6. Error Handling & Notifications

- Implement error handling and logging
- Set up push notifications for transcription status

7. Testing & QA

- Write unit and integration tests
- Conduct user acceptance testing (UAT)
- Fix bugs and improve based on feedback

8. Deployment & Scaling

- Set up hosting and deployment (CI/CD pipeline)
- Monitor performance and optimize for scaling

9. Post-Launch Iteration

- Collect user feedback

Member 1: Backend Developer

1. Set up version control and backend environment
2. Implement audio file upload and storage API
3. Integrate OpenAI Whisper for transcription
4. Handle error logging and backend API functionality

Member 2: Frontend Developer

5. Set up frontend project and dependencies
6. Build UI for audio upload, transcription display, and export
7. Implement editing features and display error messages

Member 3: Full-stack Developer

8. Set up user authentication (login/signup)
9. Implement user profiles and database for transcription storage
10. Integrate cloud storage for large files

Member 4: DevOps/Infrastructure

11. Set up CI/CD pipeline and deployment process
12. Configure hosting and auto-scaling
13. Implement performance monitoring tools

Member 5: QA/Tester

14. Write unit and integration tests for both frontend and backend
15. Conduct user acceptance testing (UAT)
16. Perform bug fixes and post-launch support

Timeline

Morning (3-4 hours)

- >**Backend Developer:** Set up version control, integrate Whisper, and create audio upload & transcription API.
- >**Frontend Developer:** Set up frontend, build UI for audio upload and display transcriptions.

Midday (2-3 hours)

- >**Full-stack Developer:** Implement user authentication, set up basic database, and cloud storage for files.
- >**DevOps:** Set up hosting and CI/CD pipeline, implement basic performance monitoring.

Afternoon (2-3 hours)

- >**QA/Tester:** Write unit tests, conduct UAT, verify audio upload and transcription functionality.

Last Hour (1 hour)

- > Collect user feedback **Team-wide Review:** Final testing and bug fixes.
-

Phase-5: Project Development

Key Points:

1 Technology Stack Used:

Core:

- **Transcription:** OpenAI Whisper API
- **Language:** Python
- **Libraries:** openai

UI (Choose one):

- **CLI:** (No framework needed, just `input()` and `print()`)
- **Desktop:** Tkinter, PyQt, or Kivy
- **Web:** Flask, Django, or FastAPI (plus React/Vue/Angular for a rich UI)
- **Mobile:** React Native, Flutter

Backend (If needed for Web/Mobile):

- **Web Server:** Gunicorn/uWSGI + Nginx/Apache
- **Database:** PostgreSQL, MySQL, SQLite, MongoDB
- **Cloud (Hosting):** AWS, GCP, Azure, Heroku

2. Development Process:

Plan: Define scope (core features, UI, output), check feasibility.

Tech Stack: Choose UI, backend, and database.

Develop (Iterative):

- **MVP:** Core transcription functionality (audio in, text out).
- **Iterate:** UI, advanced features, backend integration.

Test: Unit, integration, end-to-end, usability, load.

Deploy (if web): Cloud platform, configure server, deploy code.

Maintain: Monitor API usage, fix bugs, gather feedback

3.Challenges & Fixes:

Challenges:

Accuracy: Mis-transcriptions (noise, accents, jargon).

File Size: Whisper's limit on audio file size.

Rate Limits/Cost: API rate limits and usage costs.

Latency: Delay in real-time transcription.

Language Support: Imperfect language/dialect support.

UI: Poor user experience.

Security: Data privacy and API key security.

Speaker Diarization: Identifying speakers.

Fixes:

Accuracy: Pre-processing audio, prompting, language specification, post-processing, human review.

File Size: Splitting audio, streaming transcription, compression.

Rate Limits/Cost: Retry logic, caching, usage monitoring, optimize API calls.

Latency: Streaming, optimize chunk size, efficient code.

Language: Language detection, thorough testing.

UI: Intuitive design, clear instructions, progress, error messages.

Security: Encryption, access control, privacy compliance, secure API key storage.

Speaker Diarization: Separate diarization tool.

Phase-6: Functional & Performance Testing

Objective:

- Ensure the project works as expected.

Key Points:

1. Test Cases Executed:

I. Functional:

Basic transcription (clean audio, different languages).

Audio format support (supported/unsupported).

Error handling (invalid API key, network errors, corrupted files).

Feature tests (prompt, language detection, splitting, live transcription).

II. Performance:

Transcription time (short/long audio, different formats).

Load test (web app, concurrent users).

Memory usage.

III. Usability:

Intuitive UI, clear display, informative errors, accessibility.

IV. Accuracy:

Noise, accents, jargon, overlapping speech, low bitrate.

V. Security:

Secure API key storage.

Web security vulnerabilities (XSS, SQL injection).

Access control, data encryption.

VI. Edge Cases:

Low volume, extended silence, sudden loud noises

Bug Fixes & Improvements:

Bugs (Fixes Needed):

- **Transcription Errors:** Mis-transcriptions, hallucinations, formatting.
- **File Issues:** Upload failures, file size limits not enforced.
- **Error Handling:** Unclear messages, unhandled API errors.
- **UI/UX:** Unresponsive UI, unreadable text, no editing.
- **Security:** Exposed API key, injection vulnerabilities.

Improvements (Enhancements):

- **Accuracy:** Pre-processing, active learning, domain-specific adaptation.
- **File Handling:** Automatic splitting, more formats, progressive upload.
- **Error Handling:** Specific messages, retry mechanism.
- **UI/UX:** Real-time transcription, rich editor, speaker diarization, keyword highlighting.
- **Features:** Translation, summarization, sentiment analysis, service integration.
- **Security:** Secure API key, input validation, data encryption.
- **Performance:** API Caching, Asynchronous processing, optimize audio encoding

Final Validation:

I. Core Functionality:

- **Transcription Accuracy:** Verify across diverse samples.
- **File Handling:** Test supported formats, max size, error handling.
- **Language:** Accuracy for all languages, automatic detection.
- **Error Handling:** Graceful errors, retry mechanism.
- **Feature Validation:** Test all features and their performance.

II. UI/UX:

- **Usability:** UAT, intuitive, responsive.
- **Accessibility:** WCAG compliance, screen readers.

- Visual Appeal: Professional look.

III. Performance:

- Response Time: Acceptable transcription speed.
- Load Testing: Scalability and stability under load.

IV. Security:

- API Key Security: Secure storage.
- Input Validation: Prevent injection attacks.
- Data Security: Encryption, access control.
- Vulnerability Scanning: Identify and fix vulnerabilities.

V. Edge Cases:

- Extreme conditions (noise, low volume), boundary conditions.

VI. Legal & Compliance:

- Terms of Service/Privacy Policy: Compliant with regulations.
- OpenAI Compliance: Adherence to OpenAI's guidelines.

VII. Deployment:

- Documented and repeatable process, correct environment variables.

VIII. Monitoring:

- Monitoring setup, proper logging.

IX. Documentation:

- Complete user and developer documentation.

Final: Stakeholder sign-off before release.

Deployment (if applicable): (Hosting details or final demo link)

Final Submission

1. **Project Report Based on the templates**
2. **Demo Video (3-5 Minutes)**
3. **GitHub/Code Repository Link**

4. Presentation
